

Question 1.

1. The heuristic used for my steps cost was to add 1 to the heuristic cost for each letter in the current state that does not match the goal state. This heuristic cost can't overestimate the actual cost because it should be the same as the actual cost.

Some pseudo code:

```
if cost == 'steps':  
    for i in range(len(state)):  
        if node[i] != goal[i]:  
            cost += 1  
    return cost
```

2. The heuristic used for my scrabble cost was to look at each letter the current state had, and if it was different from the goal state, add the cost of changing that letter into the correct letter. This heuristic could not overestimate the cost because it counted the lowest possible cost to get from the current state to the goal.

Some pseudo code:

```
if self.cost == 'scrabble':  
    for i in range(len(node.state)):  
        if node.state[i] != self.goal[i]:  
            cost += cost(self.goal[i])  
    return cost
```

3. The heuristic used for my frequency cost was to return the frequency functions rarity cost of the goal. This method could not overestimate the cost because it took the rarity cost of the current state to the goal state, even though the actual cost is the cost of all the word states used to get to the goal.

Some pseudo code:

```
if self.cost == 'frequency':  
    cost += dictionary[node.state] + 1  
    return cost
```

Question 2.

Word pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY → ASK	Steps	5	773	5	16	97.9%
WHY → ASK	Scrabble	9	765	9	13	98.3%
WHY → ASK	Frequency	38.86	808	38.86	616	23.8%
BEAR → DUCK	Steps	4	691	4	6	99.1%
BEAR → DUCK	Scrabble	11	2981	6	11	99.6%
BEAR → DUCK	Frequency	38.24	1084	38.24	875	19.3%

1. UCS and A* had a different cost for bear -> duck using the frequency cost. This could be because UCS might not have expanded the path with the actual cheapest cost, while A* did due to the added heuristic function.
2. A* produced the largest percentage reduction for scrabble cost on both word pairs.
3. The heuristic was least informative for frequency for A* for both word pairs, but least informative on frequency for why -> ask and on scrabble for bear -> duck for UCS.
4. The difference is probably because UCS only takes into account the actual cost from one node to the next, while A* takes into account the actual cost and the heuristic cost. UCS is looking for the cheapest actual cost in the current moment while A* is looking for the cheapest path in regard to the heuristic and the actual cost. With a good heuristic, A* is able to expand less nodes.

Question 3.

For the steps cost measure, the cost from $w_1 \rightarrow w_2$ will always be the same as $w_2 \rightarrow w_1$. This is because the cost to transition from each letter is the same and the same word path can be used to get from one word to the other.

For the scrabble cost measure, the cost from $w_1 \rightarrow w_2$ will not always be the same as $w_2 \rightarrow w_1$. This is because the cost to transition from each letter is not the same. Changing different letters can result in a different cost. An example of this can be seen when going from bear $\rightarrow$ duck where the cost is 11, but the cost from duck $\rightarrow$ bear is 6.

For the frequency cost measure, the cost from $w_1 \rightarrow w_2$ will not always be the same as $w_2 \rightarrow w_1$. This is because the rarity cost of each word is not the same. This change can cause a different path to be taken, depending on what the algorithm sees as a cheaper path from the starting word. An example of this can be seen when going from bear $\rightarrow$ duck where the cost is 38.24, but the cost from duck $\rightarrow$ bear is 36.79.

Question 4.

1. Claud sonnet 5 was used. This was the prompt given:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary.

The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal.

By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

The proposed heuristic was :

Setup

Fix the instance's word length $L \in \{3, 4\}$, let D_L be the set of dictionary words of length L , and let **goal** be the target word. Write

$$h_0(n) = \text{Hamming}(n, \text{goal})$$

for the baseline heuristic, and recall the cost of an action that *enters* word w is $c(w) = 1 + \text{rarity}(w)$, with $\text{rarity}(w) \geq 0$.

h_0 is admissible under this cost model (each action costs ≥ 1 , and you need $\geq h_0(n)$ actions), but it throws away all the rarity information — it treats every edge as if it cost exactly 1. We can tighten it using two cheap, precomputable facts about the dictionary.

Two structural facts

Fact 1 (step-count lower bound). Changing one letter changes the Hamming distance to **goal** by at most 1 in either direction (a matched position, once changed, must become mismatched; a mismatched position may become matched or stay mismatched). So any action sequence from n to **goal** has length $k \geq h_0(n)$.

Fact 2 (the last word is known exactly). For any path ending at **goal**, the last action's destination is **goal** itself, so its cost is exactly $1 + \text{rarity}(\text{goal})$ — not just bounded below, but known.

Now define, once per instance,

$$r_{\min} = \min_{w \in D_L} \text{rarity}(w) \ (\geq 0),$$

the cheapest possible "entering cost surcharge" anywhere in the relevant part of the dictionary. This is an $O(|D_L|)$ preprocessing step, done once before search starts.



2. The formula of the heuristic:

Proposed heuristic

$$h_1(n) = \begin{cases} 0 & n = \text{goal} \\ (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(\text{goal})) & n \neq \text{goal} \end{cases}$$

3. Proof of admissibility:

Admissibility

Let $w_1, \dots, w_k = \text{goal}$ be the words entered along *any* path from n to **goal** ($k \geq h_0(n)$, by Fact 1). Its cost is

$$\sum_{j=1}^k c(w_j) = \sum_{j=1}^{k-1} c(w_j) + c(\text{goal}) \geq (k-1)(1 + r_{\min}) + (1 + \text{rarity}(\text{goal})),$$

since each w_j ($j < k$) satisfies $\text{rarity}(w_j) \geq r_{\min}$. Because $k-1 \geq h_0(n) - 1$ and $(1 + r_{\min}) \geq 0$,

$$\text{cost} \geq (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(\text{goal})) = h_1(n).$$

This holds for every path, hence in particular for the optimal one, so $h_1(n) \leq h^*(n)$. ■

(A short case check also shows h_1 is **consistent**: on an edge $n \rightarrow n'$ that reduces h_0 by 1 with $n' = \text{goal}$, $h_1(n) = c(n') + h_1(n')$ exactly; on all other edges the inequality $h_1(n) \leq c(n') + h_1(n')$ holds with slack from $\text{rarity}(n') \geq r_{\min}$. Consistency means A* with h_1 never needs to reopen a closed node.)

4.

Proposed heuristic

$$h_1(n) = \begin{cases} 0 & n = \text{goal} \\ (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(\text{goal})) & n \neq \text{goal} \end{cases}$$

5. The images below prove the heuristic using the example from question 2.

Checking admissibility against the true optimal cost

I computed the actual optimal remaining cost $h^*(n)$ by Dijkstra over the real word graph (edges = single-letter changes present in the dictionary, edge cost = $1 + \text{rarity}(\text{destination})$):

node	h_0	h_1	h^* (true optimal)	$h_1 \leq h^*$?
why	3	7.2651	38.8554	✓
bear	4	12.2842	38.2402	✓

The true optimal costs are large because these dictionaries have some very rare intermediate words unavoidable on any bridging path (e.g. the cheapest why→ask route passes through ais at rarity 13.89) — but h_1 never overshoots, as guaranteed by the proof.

Exactness right next to the goal

This is the clearest evidence the heuristic is doing real work, not just adding a constant. Along the optimal path, the word immediately before the goal has $h_0 = 1$:

node	h_0	h_1	h^*	gap
ass (before ask)	1	5.2326	5.2326	0 — exact
buck (before duck)	1	8.2297	8.2297	0 — exact

h_0 would say "1" at both nodes; h_1 correctly predicts the true remaining cost to the decimal, because at $h_0 = 1$ the formula reduces to exact' + rarity(goal) — no estimation involved.

The two real instances

	L	$r_{\min}$ (min rarity, same length)	rarity(goal)
why → ask	3	0.016226 (word: the)	rarity(ask) = 4.232644
bear → duck	4	0.351506 (word: that)	rarity(duck) = 7.229719

h_1 at the start states

$$h_1(n) = (h_0(n) - 1)(1 + r_{\min}) + (1 + \text{rarity}(\text{goal}))$$

- **why**: $h_0 = 3$, so $h_1(\text{why}) = (2)(1.016226) + (5.232644) = \mathbf{7.2651}$ — versus $h_0 = 3$.
- **bear**: $h_0 = 4$, so $h_1(\text{bear}) = (3)(1.351506) + (8.229719) = \mathbf{12.2842}$ — versus $h_0 = 4$.

Both cases: $h_1 > h_0$ by a wide margin, since these dictionaries have real, substantial rarity spread.